

タスク管理ツール バックエンド・コードガイド

設計書を Python コードへ変換するための、役割別サンプルと読み方

STACK	FastAPI / SQLAlchemy 2.x / Pydantic v2 / MySQL 8.0
-------	--

API	http://localhost:8888
-----	-----------------------

DATABASE	localhost:13306 / test_db
----------	---------------------------

DATE	2026-09-09
------	------------

この PDF のコードは役割を理解するための短い例です。docs/backend_design.md を正とし、各 TODO ファイルへ自分で組み立ててください。

00 このガイドの使い方

バックエンド全体を一度に書かず、DB 接続から API までを層ごとに積み上げます。各章の「入力」「処理」「出力」を意識すると、どこで値が変わるか追いやすくなります。

順番	章	実装先	到達点
1	01-03	config / database / models	MySQL に3テーブルが作られる
2	04-05	schemas / dependencies	入力検証と1リクエスト1Session
3	06	category / assignee router	参照データを GET / POST できる
4	07-09	task CRUD / router	CRUD・絞り込み・関連の入れ子
5	10	main	CORS と router を統合して起動
6	11-13	frontend / test	画面接続と受け入れ確認

設計書と PDF の役割

設計書はフィールド名・制約・ステータスコードを決める「正解表」です。この PDF は、その設計を FastAPI / SQLAlchemy のコードへ写す考え方を示します。例文と設計が違って見えたら、設計書を優先してください。

最初に覚えるデータの往復

HTML	JavaScript	FastAPI	SQLAlchemy	MySQL
select.value = "2"	category_id = 2	Pydantic 検証	Task.category_id	tasks 行
category.name を表示	JSON を state へ	TaskResponse	relationship	JOIN 対象

- 写経だけで進めず、各関数がどの層へ値を渡すかを1行ずつ確認する。
- 最初は /docs で API 単体を確認し、最後に frontend をつなぐ。
- POST / PUT / DELETE の直後に GET し、DB へ保存された結果で画面を更新する。

01 コードを役割ごとに分ける

1つの main.py に全部を書くこともできますが、課題全体では「設定」「DB」「モデル」「入出力」「HTTP」「DB操作」を分離した方が、エラーの場所を特定しやすくなります。

層	主なファイル	入力	出力 / 次へ渡すもの
設定	config.py	.env	settings オブジェクト
DB 基盤	database.py	settings	engine / SessionLocal / Base
保存形式	models.py	Base	ORM クラスと relationship
API 形式	schemas.py	JSON / ORM	検証済み入力 / JSON 出力
Session	dependencies.py	SessionLocal	各 request 専用 Session
DB 操作	crud/*.py	Session + 値	ORM オブジェクト
HTTP	routers/*.py	path / query / body	status + response model
統合	main.py	全 router / middleware	FastAPI app

import の向き

依存方向の目安

```
main.py
-> routers/tasks.py
    -> crud/tasks.py
        -> models.py
        -> database.py
    -> schemas.py
    -> dependencies.py
        -> database.py

config.py -> database.py -> models.py
```

循環 import を避ける

models.py が router を import するような逆向き依存を作らないこと。Base.metadata.create_all() 前にモデル登録が必要なため、main.py 側で models を一度読み込む形にします。

同期 DB を使うルート

この課題は PyMySQL と同期版 SQLAlchemy Session を使います。DB ルートは def で書くと、FastAPI が同期処理用のスレッドで実行します。async def の中で同期 DB を直接待つ構成は避けます。

02 設定と DB 接続を作る

接続値をコードへ直書きせず `.env` へ置きます。`config.py` は値の読込、`database.py` は SQLAlchemy 部品の作成だけを担当します。

app/config.py - 例

```
from functools import lru_cache
from sqlalchemy import URL
from pydantic_settings import BaseSettings, SettingsConfigDict

class Settings(BaseSettings):
    db_host: str
    db_port: int
    db_name: str
    db_user: str
    db_password: str
    db_charset: str = "utf8mb4"
    cors_origins: str

    model_config = SettingsConfigDict(
        env_file=".env", env_file_encoding="utf-8", extra="ignore"
    )

    @property
    def database_url(self) -> URL:
        return URL.create(
            "mysql+pymysql",
            username=self.db_user,
            password=self.db_password,
            host=self.db_host,
            port=self.db_port,
            database=self.db_name,
            query={"charset": self.db_charset},
        )

    @property
    def allowed_origins(self) -> list[str]:
        return [value.strip() for value in self.cors_origins.split(",") if value.strip()]

@lru_cache
def get_settings() -> Settings:
    return Settings()
```

app/database.py - 例

```
from sqlalchemy import create_engine
from sqlalchemy.orm import DeclarativeBase, sessionmaker
from .config import get_settings

settings = get_settings()

engine = create_engine(
    settings.database_url,
    pool_pre_ping=True,
    pool_recycle=3600,
    echo=False,
)

SessionLocal = sessionmaker(
    bind=engine,
    autoflush=False,
    expire_on_commit=False,
)

class Base(DeclarativeBase):
    pass
```

ポート 13306 の意味

Docker 内の MySQL は 3306 ですが、ホスト PC には 13306 として公開されています。FastAPI もホストで起動するため localhost:13306 を使います。FastAPI まで Docker 化した場合だけ接続先の考え方が変わります。

接続だけを先に確認する

モデルを書く前に、短い一時コードで `engine.connect()` が成功するか確認すると、接続失敗とモデル失敗を分離できます。接続 URL 全体はパスワードを含むため、そのまま print しません。

backend/.env - ローカル学習環境

```
DB_HOST=localhost
DB_PORT=13306
DB_NAME=test_db
DB_USER=test_user
DB_PASSWORD=test_password
DB_CHARSET=utf8mb4
CORS_ORIGINS=http://localhost:3000,http://127.0.0.1:3000
```

接続時の症状	確認する値	判断
Connection refused	Docker logs / DB_PORT	MySQL ready と 13306 を確認
Access denied	DB_USER / DB_PASSWORD	test_user / test_password
Unknown database	DB_NAME	test_db。既存 volume の初期値にも注意
日本語が文字化け	DB_CHARSET / 接続 URL	utf8mb4 を query に含める

03 ORM モデルで3テーブルを表す

ORM モデルは DB の保存形式です。API の response と同じ形にする必要はありません。Task は category_id を保存し、relationship の category 経由で名前を取得します。

Model	必須列	任意列	関係
Category	id, name	なし	tasks $\times$ 1:N
Assignee	id, name	なし	tasks $\times$ 1:N
Task	id, title, is_done, created_at, updated_at	description, category_id, assignee_id	category / assignee $\times$ N:1

app/models.py - Category / Assignee の例

```
from datetime import datetime
from sqlalchemy import Boolean, DateTime, ForeignKey, String, Text, func, text
from sqlalchemy.orm import Mapped, mapped_column, relationship
from .database import Base

class Category(Base):
    __tablename__ = "categories"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100), unique=True, nullable=False)
    tasks: Mapped[list["Task"]] = relationship(
        back_populates="category", passive_deletes=True
    )

class Assignee(Base):
    __tablename__ = "assignees"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(100), unique=True, nullable=False)
    tasks: Mapped[list["Task"]] = relationship(
        back_populates="assignee", passive_deletes=True
    )
```

03 ORM モデルで3テーブルを表す（続き）

app/models.py - Task の例

```
class Task(Base):
    __tablename__ = "tasks"

    id: Mapped[int] = mapped_column(primary_key=True)
    title: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[str | None] = mapped_column(Text, nullable=True)
    is_done: Mapped[bool] = mapped_column(
        Boolean, nullable=False, default=False, server_default=text("0"), index=True
    )
    category_id: Mapped[int | None] = mapped_column(
        ForeignKey("categories.id", ondelete="SET NULL"), nullable=True, index=True
    )
    assignee_id: Mapped[int | None] = mapped_column(
        ForeignKey("assignees.id", ondelete="SET NULL"), nullable=True, index=True
    )
    created_at: Mapped[datetime] = mapped_column(
        DateTime, nullable=False, server_default=func.now()
    )
    updated_at: Mapped[datetime] = mapped_column(
        DateTime, nullable=False, server_default=func.now(), onupdate=func.now()
    )

    category: Mapped[Category | None] = relationship(back_populates="tasks")
    assignee: Mapped[Assignee | None] = relationship(back_populates="tasks")
```

指定	役割	この課題での理由
nullable=True	NULL を許可	カテゴリ・担当者の未設定を表す
ForeignKey	DB でも参照整合性を守る	存在しない ID を保存しない
relationship	ORM オブジェクト間を移動	response の category / assignee を作る
unique=True	同名を DB で拒否	画面の選択肢重複を防ぐ
index=True	検索用索引	is_done / category_id の絞り込み
server_default	DB 側初期値	作成時刻と false を確実に入れる
onupdate	更新時刻を変更	PUT 後の updated_at を新しくする

create_all の限界

Base.metadata.create_all() は未作成テーブルを作りますが、すでにあるテーブルの列変更は行いません。モデルを直したのに DB が変わらない場合、学習用 DB を安全に初期化するか migration を使う必要があります。

04 Pydantic で API の形を固定する

Pydantic スキーマはブラウザとの契約です。Create、Update、Response を分けると、入力 ID と出力オブジェクトの違いを明確にできます。

app/schemas.py - 参照データの例

```
from datetime import datetime
from typing import Annotated
from pydantic import BaseModel, ConfigDict, Field, field_validator

Name = Annotated[str, Field(min_length=1, max_length=100)]
Title = Annotated[str, Field(min_length=1, max_length=255)]
PositiveId = Annotated[int, Field(gt=0)]

class CategoryCreate(BaseModel):
    name: Name

    @field_validator("name", mode="before")
    @classmethod
    def strip_name(cls, value):
        return value.strip() if isinstance(value, str) else value

class CategoryResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    name: str

class AssigneeCreate(BaseModel):
    name: Name

    @field_validator("name", mode="before")
    @classmethod
    def strip_name(cls, value):
        return value.strip() if isinstance(value, str) else value

class AssigneeResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    name: str
```

app/schemas.py - Task の例

```
class TaskBase(BaseModel):
    title: Title
    description: str | None

    @field_validator("title", mode="before")
    @classmethod
    def strip_title(cls, value):
        return value.strip() if isinstance(value, str) else value

class TaskCreate(TaskBase):
    description: str | None = None
    category_id: PositiveId | None = None
    assignee_id: PositiveId | None = None

class TaskUpdate(TaskBase):
    is_done: bool
    category_id: PositiveId | None
    assignee_id: PositiveId | None

class TaskResponse(BaseModel):
    model_config = ConfigDict(from_attributes=True)
    id: int
    title: str
    description: str | None
    is_done: bool
    category: CategoryResponse | None
    assignee: AssigneeResponse | None
    created_at: datetime
    updated_at: datetime
```


None を許すことと、省略を許すことは別

TaskUpdate の description: str | None にはデフォルト値を書いていません。そのため JSON の値は null にできますが、フィールド自体は必須です。PUT を全項目更新として扱う重要な設計です。

request と response を分ける理由

場面	Category / Assignee	日時	is_done
POST request	category_id / assignee_id	送らない	送らない (False 固定)
PUT request	category_id / assignee_id	送らない	必須
Task response	category / assignee オブジェクト	created_at / updated_at	必須

422 response の読み方 (例)

```
{
  "detail": [
    {
      "loc": ["body", "title"],
      "msg": "String should have at least 1 character",
      "type": "string_too_short"
    }
  ]
}
```

loc はエラー位置、msg は理由です。frontend/js/api.js はこの配列を1つの読みやすいメッセージへ変換します。

05 DB Session をリクエスト単位で渡す

Session は DB との作業単位です。グローバルな1個を使い回さず、FastAPI の Depends で各 HTTP リクエストへ新しい Session を渡します。

app/dependencies.py - 例

```
from collections.abc import Generator
from typing import Annotated
from fastapi import Depends
from sqlalchemy.orm import Session
from .database import SessionLocal

def get_db() -> Generator[Session, None, None]:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

DbSession = Annotated[Session, Depends(get_db)]
```

ルーターで受け取る

Annotated alias を使う例

```
@router.get("", response_model=list[CategoryResponse])
def list_categories(db: DbSession):
    ...
```

タイミング	Session の状態	担当
リクエスト開始	SessionLocal() で生成	get_db
router / CRUD	同じ Session を共有	Depends
成功更新	commit する	更新処理
失敗更新	rollback する	例外を捕捉した処理
レスポンス後	必ず close	get_db の finally

commit は get_db に隠さない

どの API がデータを確定するか見えるよう、POST / PUT / DELETE の処理側で commit します。GET は commit しません。例外時に rollback した Session は、次の DB 操作へ再利用しない方が追いやすくなります。

06 カテゴリと担当者 API を先に完成させる

Task より構造が単純な GET / POST から始めます。Category を完成させたら、同じ構造を Assignee へ展開します。

Category router の短い実装例

```
from fastapi import APIRouter, HTTPException, status
from sqlalchemy import select
from sqlalchemy.exc import IntegrityError
from .dependencies import DbSession
from .models import Category
from .schemas import CategoryCreate, CategoryResponse

router = APIRouter(prefix="/categories", tags=["categories"])

@router.get("", response_model=list[CategoryResponse])
def list_categories(db: DbSession):
    statement = select(Category).order_by(Category.id)
    return db.scalars(statement).all()

@router.post(
    "", response_model=CategoryResponse,
    status_code=status.HTTP_201_CREATED,
)
def create_category(payload: CategoryCreate, db: DbSession):
    category = Category(name=payload.name)
    db.add(category)
    try:
        db.commit()
    except IntegrityError:
        db.rollback()
        raise HTTPException(
            status_code=status.HTTP_409_CONFLICT,
            detail="Category name already exists",
        )
    db.refresh(category)
    return category
```

Assignee へ写すときに変える箇所

Category 側	Assignee 側
/categories	/assignees
Category	Assignee
CategoryCreate / Response	AssigneeCreate / Response
Category name already exists	Assignee name already exists
categories tag	assignees tag

事前検索だけでは重複を完全に防げない

同時に2件の POST が来ると、両方の事前検索が「なし」になる可能性があります。分かりやすい事前確認に加えて、DB の UNIQUE 制約と IntegrityError を最終防衛線にします。

07 Task の取得と絞り込み

一覧 API は query が未指定でも指定済みでも同じ関数を使います。false を正しく扱い、関連をまとめて読み込むことが要点です。

app/crud/tasks.py - 読み取り例

```
from sqlalchemy import select
from sqlalchemy.orm import selectinload
from .models import Task

TASK_WITH_RELATIONS = (
    select(Task)
    .options(
        selectinload(Task.category),
        selectinload(Task.assigee),
    )
)

def find_tasks(db, *, is_done=None, category_id=None):
    statement = TASK_WITH_RELATIONS

    if is_done is not None:
        statement = statement.where(Task.is_done == is_done)

    if category_id is not None:
        statement = statement.where(Task.category_id == category_id)

    statement = statement.order_by(Task.created_at.desc(), Task.id.desc())
    return db.scalars(statement).all()

def find_task(db, task_id: int):
    statement = TASK_WITH_RELATIONS.where(Task.id == task_id)
    return db.scalar(statement)
```

app/routers/tasks.py - GET 例

```
router = APIRouter(prefix="/tasks", tags=["tasks"])

@router.get("", response_model=list[TaskResponse])
def list_tasks(
    db: DbSession,
    is_done: bool | None = None,
    category_id: int | None = None,
):
    return find_tasks(db, is_done=is_done, category_id=category_id)

@router.get("/{task_id}", response_model=TaskResponse)
def get_task(task_id: int, db: DbSession):
    task = find_task(db, task_id)
    if task is None:
        raise HTTPException(status_code=404, detail="Task not found")
    return task
```

最重要: false は値である

if is_done: と書くと false 指定時に条件が追加されません。is_done is not None と比較し、「未指定」と「false」を分けてください。該当なしの一覧は 404 ではなく [] です。

08 Task を作成・更新する

外部キー ID は、Task へ保存する前に存在確認します。POST では is_done=False、PUT では受け取った5項目をすべて更新します。

POST /tasks - 例

```
def require_reference(db, model, value, label):
    if value is not None and db.get(model, value) is None:
        raise HTTPException(status_code=404, detail=f"{label} not found")

@router.post(
    "", response_model=TaskResponse,
    status_code=status.HTTP_201_CREATED,
)
def create_task(payload: TaskCreate, db: DbSession):
    require_reference(db, Category, payload.category_id, "Category")
    require_reference(db, Assignee, payload.assignee_id, "Assignee")

    task = Task(
        **payload.model_dump(),
        is_done=False,
    )
    db.add(task)
    try:
        db.commit()
    except SQLAlchemyError:
        db.rollback()
        raise

    return find_task(db, task.id)
```

PUT /tasks/{task_id} - 例

```
@router.put("/{task_id}", response_model=TaskResponse)
def update_task(task_id: int, payload: TaskUpdate, db: DbSession):
    task = db.get(Task, task_id)
    if task is None:
        raise HTTPException(status_code=404, detail="Task not found")

    require_reference(db, Category, payload.category_id, "Category")
    require_reference(db, Assignee, payload.assignee_id, "Assignee")

    for field, value in payload.model_dump().items():
        setattr(task, field, value)

    try:
        db.commit()
    except SQLAlchemyError:
        db.rollback()
        raise

    return find_task(db, task.id)
```

本番を意識するなら、想定外の SQLAlchemyError をログへ記録し、接続情報を含まない一般的な 500 応答へ変換します。学習中はまず例外を隠しすぎず、サーバー側の traceback で原因を確認してください。

09 204 とステータスコードを仕様どおり返す

正しい JSON だけでなく、HTTP status も API 契約です。POST は 201、DELETE 成功は 204 で本文を返しません。

DELETE /tasks/{task_id} - 例

```
from fastapi import Response, status

@router.delete(
   ("/{task_id}",
    status_code=status.HTTP_204_NO_CONTENT,
    response_class=Response,
)
def delete_task(task_id: int, db: DbSession) -> Response:
    task = db.get(Task, task_id)
    if task is None:
        raise HTTPException(status_code=404, detail="Task not found")

    db.delete(task)
    try:
        db.commit()
    except SQLAlchemyError:
        db.rollback()
        raise

    return Response(status_code=status.HTTP_204_NO_CONTENT)
```

API	成功	存在しない	入力不正 / 重複
GET list	200 + 配列	200 + []	422 query 不正
GET one	200 + object	404	422 path 不正
POST	201 + object	関連 ID は 404	422 / 409
PUT	200 + object	404	422
DELETE	204 + 空本文	404	422 path 不正

フロントエンド側の注意

204 は本文がないため `response.json()` を呼ぶと失敗します。frontend/js/api.js の共通 `request()` は `status === 204` のとき `null` を返し、それ以外だけ JSON を解析します。

更新日時を確認する

PUT の前後で `updated_at` が変わること、`created_at` は変わらないことを /docs で確認します。モデルの `onupdate` だけで反映されない構成なら、更新時に明示的に現在時刻を設定する案もあります。

10 main.py で統合し、CORS を許可する

最後に、テーブル作成・CORS・3 router を FastAPI アプリへ集めます。models が metadata に登録された後で create_all を呼びます。

app/main.py - 統合例

```
from contextlib import asynccontextmanager
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

from . import models as _models
from .config import get_settings
from .database import Base, engine
from .routers import assignees, categories, tasks

@asynccontextmanager
async def lifespan(app: FastAPI):
    Base.metadata.create_all(bind=engine)
    yield

settings = get_settings()
app = FastAPI(title="Task Manager API", lifespan=lifespan)

app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.allowed_origins,
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(tasks.router)
app.include_router(categories.router)
app.include_router(assignees.router)
```

backend フォルダで実行

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8888
```

URL	用途
http://localhost:8888/docs	Swagger UI で API 単体確認
http://localhost:3000	フロントエンド
http://127.0.0.1:3000	同じ PC でも別 origin。両方許可
localhost:13306	FastAPI から MySQL への接続

CORS エラーの見分け方

Swagger UI は API と同じ origin なので、CORS が間違っていないでも動きます。/docs で成功し、ブラウザ画面だけ失敗する場合は DevTools の Console と Network で Origin / OPTIONS / Access-Control-Allow-Origin を確認します。

11 フロントエンドとのつながりを読む

完成済み frontend は、通信だけを `api.js`、画面状態と DOM 更新を `app.js` に分けています。バックエンド実装時は Network タブで request と response を照合します。

UI 操作	api.js	FastAPI	画面更新
初期表示	<code>getTasks / getCategories / getAssignees</code>	3つの GET	state -> 一覧 / select
タスク追加	<code>createTask</code>	POST /tasks	フォーム reset -> GET
チェック	<code>updateTask</code>	PUT /tasks/{id}	GET してカード更新
編集を開く	<code>getTask</code>	GET /tasks/{id}	edit dialog に設定
編集保存	<code>updateTask</code>	PUT /tasks/{id}	dialog close -> GET
削除	<code>deleteTask</code>	DELETE /tasks/{id}	204 -> GET
Category 追加	<code>createCategory</code>	POST /categories	3 select へ反映
Assignee 追加	<code>createAssignee</code>	POST /assignees	2 select へ反映

response -> PUT request の変換

```
// GET response has nested objects.
task.category      // { id: 2, name: "Work" }
task.assignee      // { id: 1, name: "Yamada" }

// PUT request must send IDs instead.
const payload = {
  title: task.title,
  description: task.description,
  is_done: true,
  category_id: task.category?.id ?? null,
  assignee_id: task.assignee?.id ?? null,
};
```

DOM の安全な更新

`app.js` は API から返った title / description / name を innerHTML に埋めず、`createElement()` と `textContent` で表示します。入力文字列を HTML として実行しないための基本的な防御です。

12 小さく動かして完成まで進める

各段階で確認可能な状態を作り、失敗範囲を狭くします。下のチェックを上から進め、合格した項目へ印を付けます。

✓	段階	確認操作	合格
A	DB 起動	docker compose up -d / logs で ready	接続拒否がない
B	設定	.env -> Settings -> URL	13306 / test_db
C	モデル	create_all 後に DB クライアント確認	3表・FK・NULL
D	Category	POST -> GET	201 / 配列
E	Assignee	POST -> GET	201 / 配列
F	Task create	関連あり / なしで POST	入れ子 response
G	Task read	list / one / filters	200 / [] / 404
H	Task update	内容・完了・関連解除	updated_at 変更
I	Task delete	存在あり / なし	204 空本文 / 404
J	SPA	追加から削除まで	reload なし / CORS なし
K	永続化	FastAPI 再起動後 GET	データが残る

提出スクリーンショット

- 01-created.png: Category / Assignee つきで追加され、一覧に見える。
- 02-done.png: チェック後に取り消し線と完了表示が見える。
- 03-edited.png: タイトルや説明を変更した結果が見える。
- 04-deleted.png: 削除後に対象が消えたことが分かる。
- 任意で /docs と DB クライアントの状態も残す。秘密情報や個人情報の写り込みを確認する。

13 詰まったときの切り分け表

エラー文だけを直そうとせず、ブラウザ、HTTP、Pydantic、SQLAlchemy、MySQL のどの境界まで成功しているかを確認します。

症状	原因候補	見る場所	確認
API に接続不可	FastAPI 停止 / port 違い	Terminal	8888 で起動
DB connection refused	MySQL 初期化中 / 3306 指定	Docker logs / .env	ready / 13306
table がない	models 未読込	main.py	create_all 前の import
422	body 名・型・必須が不一致	Network / response detail	schema と payload
Category not found	文字列 ID / 古い ID	Request Payload / DB	Number 変換と行の存在
category が null	FK 未保存 / relationship 未読込	DB / SQL log	category_id と eager load
false filter 不動	真偽値の条件分岐	crud/tasks.py	is not None
DELETE 後 JSON error	204 を JSON 解析	frontend/api.js	status === 204
ブラウザだけ失敗	CORS origin 不一致	Console / OPTIONS	localhost と 127.0.0.1
モデル修正が DB に反映されない	create_all は migration しない	DB schema	安全な再作成 / Alembic
重複 POST で 500	IntegrityError 未処理	server traceback	rollback -> 409

調査の基本順

1) Browser Network の URL / method / payload / status、2) FastAPI terminal の traceback、3) /docs で同じ request、4) DB の実データ、の順で確認すると、画面・API・DB のどこで崩れたかを分離できます。

完成前セルフレビュー

- API フィールド名が is_done / category_id / assignee_id のままか。
- response が category / assignee の {id, name} または null か。
- POST 201、DELETE 204、未存在 404 を /docs で確認したか。
- PUT は5項目を受け、null で関連解除できるか。
- DB 再接続後も Session を閉じ、失敗後に rollback しているか。
- FastAPI 再起動後もデータが残り、画面操作で CORS エラーがないか。

14 参考資料と設計上の根拠

このガイドは課題仕様を最優先にし、ライブラリ固有の書き方は公式資料で確認しています。URL は将来更新される可能性があります。

1. MySQL-Sample

Docker Compose、test_db / test_user、13306:3306

<https://github.com/pygmalin-info/MySQL-Sample>

2. FastAPI - Request Body

Pydantic model を request body として受ける

<https://fastapi.tiangolo.com/tutorial/body/>

3. FastAPI - Body Updates

PUT を全置換として扱う考え方

<https://fastapi.tiangolo.com/tutorial/body-updates/>

4. FastAPI - CORS

CORSMiddleware と origin の定義

<https://fastapi.tiangolo.com/tutorial/cors/>

5. FastAPI - Path Operation Configuration

201 などの status_code

<https://fastapi.tiangolo.com/tutorial/path-operation-configuration/>

6. FastAPI - Dependencies

DB Session を Depends で共有する考え方

<https://fastapi.tiangolo.com/tutorial/dependencies/>

7. SQLAlchemy 2.0 - ORM Quick Start

Declarative model、Session、create_all

<https://docs.sqlalchemy.org/en/20/orm/quickstart.html>

8. SQLAlchemy 2.0 - Engine

mysql+pymysql URL

<https://docs.sqlalchemy.org/en/20/core/engines.html>

9. SQLAlchemy 2.0 - Relationships

relationship の構成

https://docs.sqlalchemy.org/en/20/orm/relationship_api.html

10. Pydantic - Models

ConfigDict(from_attributes=True)

<https://docs.pydantic.dev/latest/concepts/models/#arbitrary-class-instances>

11. Pydantic Settings

.env / BaseSettings

https://docs.pydantic.dev/latest/concepts/pydantic_settings/

次に行うこと

README の順番で MySQL と仮想環境を準備し、backend/app/config.py から実装を始めます。各 Phase が動いたら commit し、最後に画面で一連の操作と再起動後の永続化を確認してください。